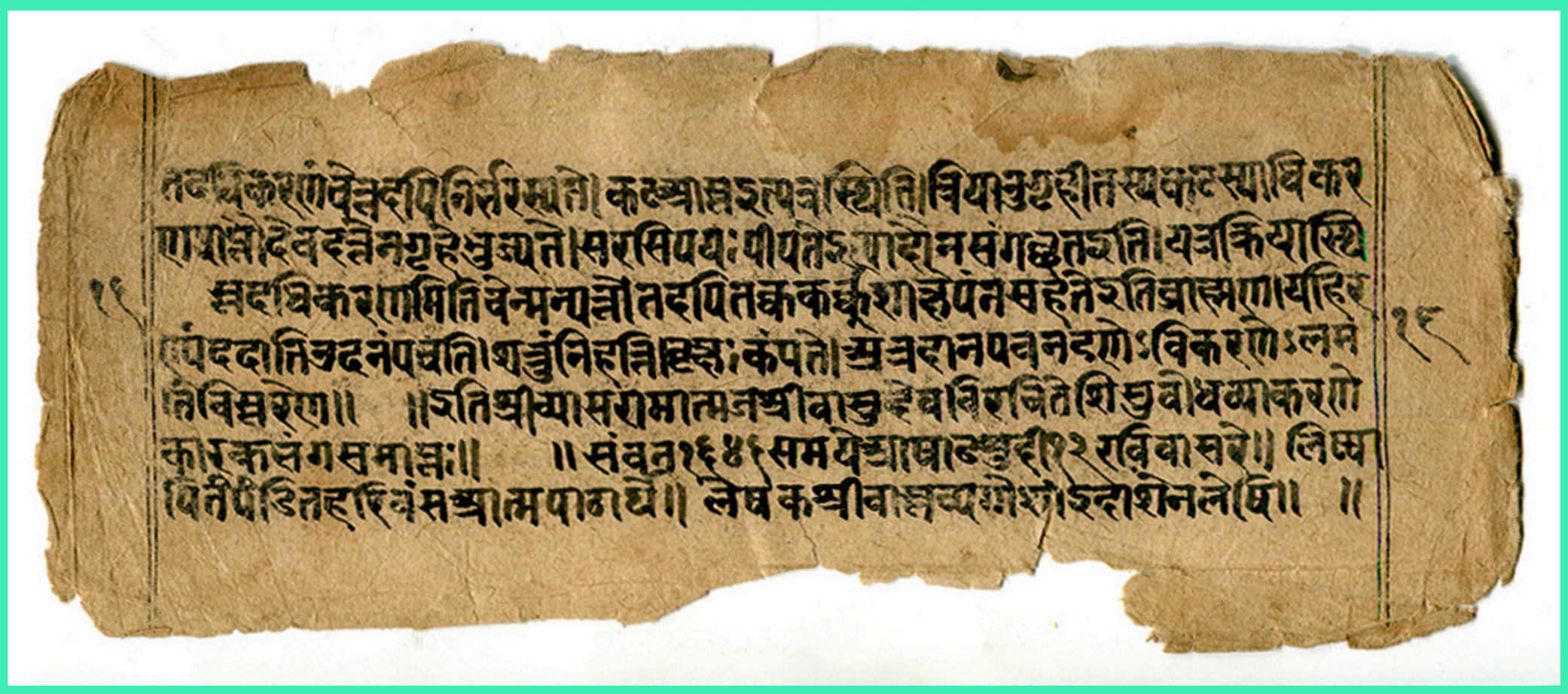
PALM LEAF MANUSCRIPT ENHANCEMENT AND LINE SEGMENTATION



PREPARED BY: MABBU KETAN PRAKASH REDDDY

M.SC BIOINFORMATICS | SVIMS

Abstract

Palm leaf manuscripts present significant challenges for digital processing due to uneven illumination, textured backgrounds, and degradation of ink over time. This project proposes a classical image processing pipeline to enhance manuscript readability and perform reliable line segmentation. The approach was developed iteratively, starting from a baseline enhancement and projection-based segmentation method, followed by systematic refinements in binarization and preprocessing. The final pipeline demonstrates that careful preprocessing—particularly binarization—plays a critical role in achieving accurate segmentation.

Introduction

Palm leaf manuscripts are historically important documents that require digitization for preservation and analysis. However, due to their physical properties, they pose several challenges:

- Non-uniform illumination
- Strong background texture
- Noise and degradation
- Irregular text alignment

A key preprocessing step is line segmentation, which separates text into individual lines for further analysis.

Objectives

To design a reproducible image processing pipeline that:

1. Enhances degraded manuscript images
2. Reduces background interference
3. Segments text lines accurately

Methodology

4.1 Attempt 1: Baseline Pipeline

- Grayscale conversion
- CLAHE (contrast enhancement)
- Bilateral filtering
- Background normalization
- Adaptive thresholding
- Morphological opening
- Horizontal projection-based segmentation

"Analysis of Line Merging Issue"

Your current logic:

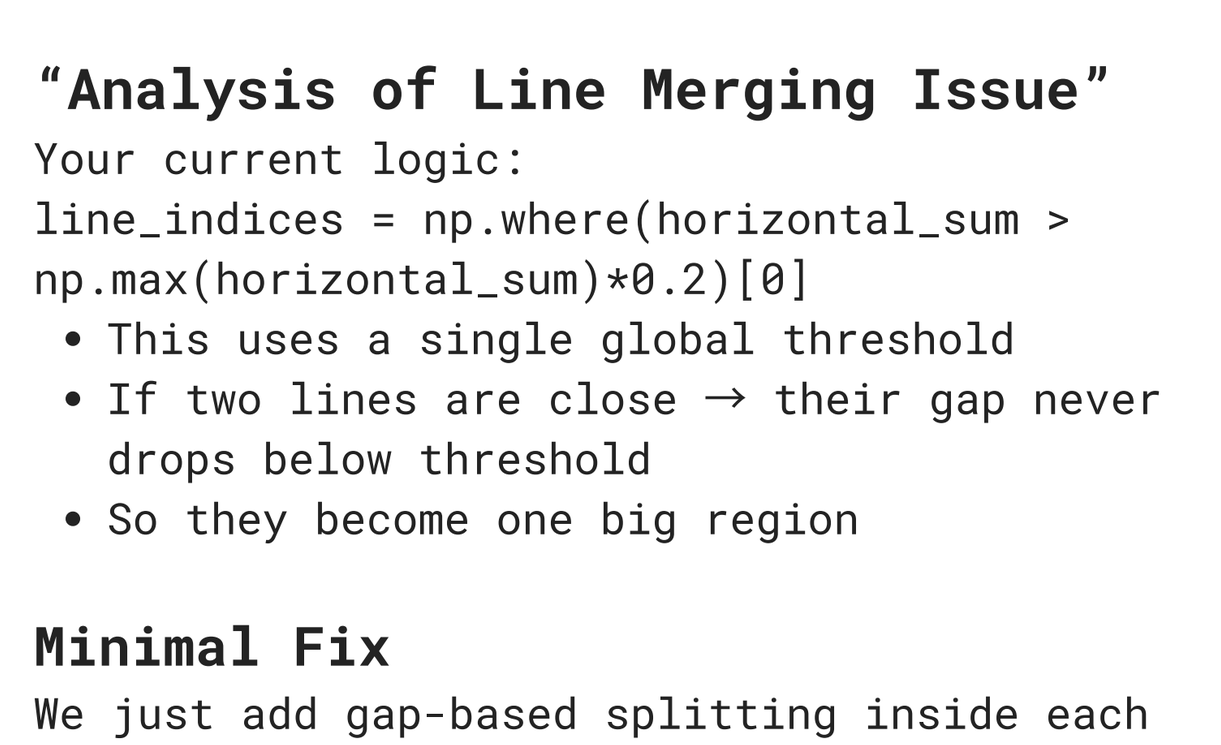
line_indices = np.where(horizontal_sum > np.max(horizontal_sum)*0.2)[0]

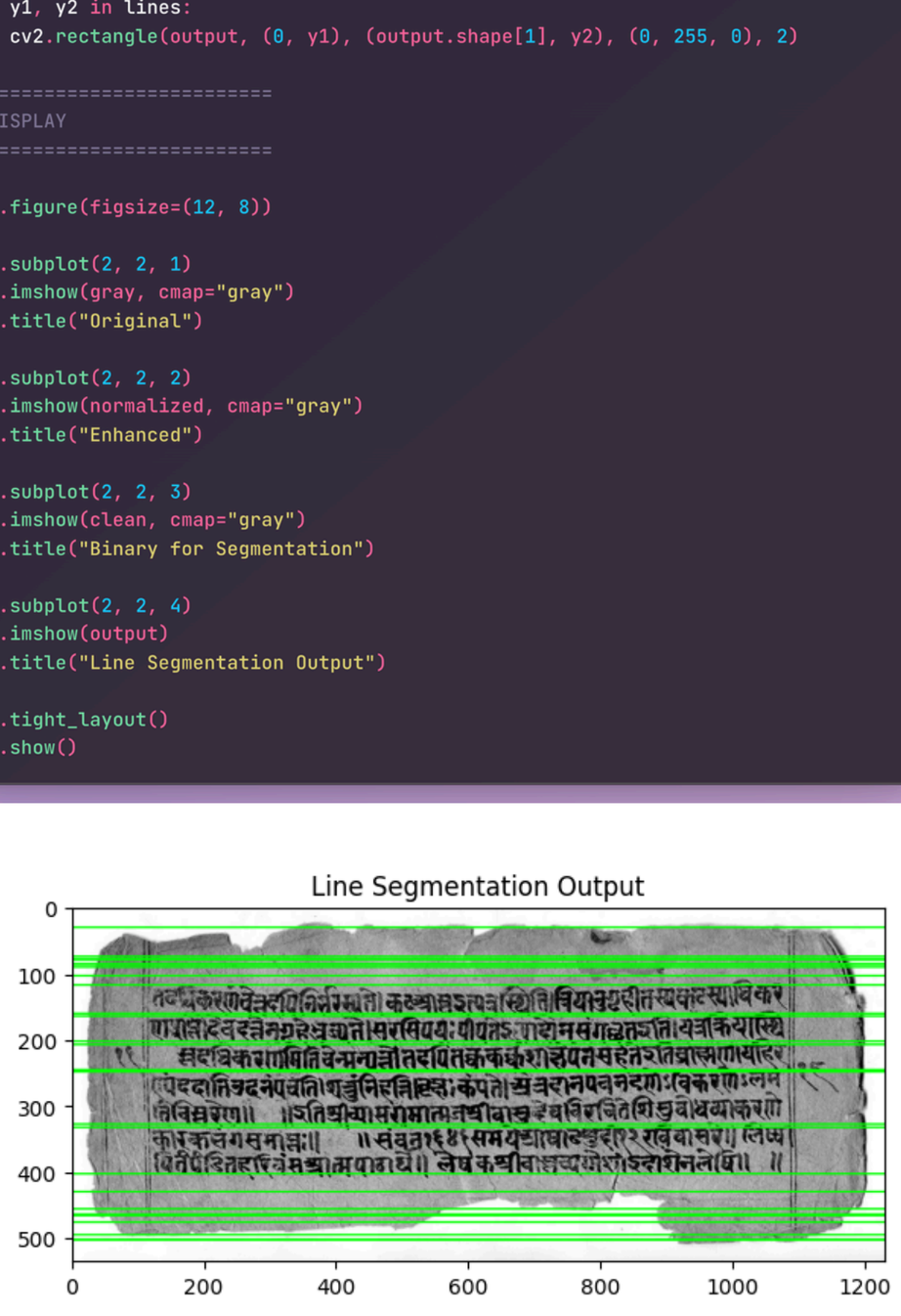
- This uses a single global threshold
- If two lines are close → their gap never drops below threshold
- So they become one big region

Minimal Fix

We just add gap-based splitting inside each detected region.

```python
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load image
img = cv2.imread("manuscript.jpg")
if img is None:
    raise ValueError("Image not loaded!")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# ========================
# STEP 1: Enhancement
# ========================

# CLAHE
clahe = cv2.createCLAHE(clipLimit=2.5, tileGridSize=(8, 8))
enhanced = clahe.apply(gray)

# Bilateral Filter
denoised = cv2.bilateralFilter(enhanced, 9, 75, 75)

# Background normalization
background = cv2.GaussianBlur(denoised, (51, 51), 0)
normalized = cv2.divide(denoised, background, scale=255)

# ========================
# STEP 2: Binarization (for segmentation)
# ========================

thresh = cv2.adaptiveThreshold(
    normalized, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY_INV, 35, 10
)

# Remove noise
kernel = np.ones((3, 3), np.uint8)
clean = cv2.morphologyEx(thresh, cv2.MORPH_OPEN, kernel)

# ========================
# STEP 3: Line Segmentation
# ========================

# Horizontal projection
horizontal_sum = np.sum(clean, axis=1)

# Threshold to detect text lines
line_indices = np.where(horizontal_sum > np.max(horizontal_sum) * 0.2)[0]

# Group into continuous lines
lines = []
start = None

for i in range(len(line_indices)):
    if start is None:
        start = line_indices[i]
    if i == len(line_indices) - 1 or line_indices[i + 1] - line_indices[i] > 2:
        end = line_indices[i]
        lines.append((start, end))
        start = None

# ========================
# STEP 4: Draw Lines
# ========================

output = cv2.cvtColor(gray, cv2.COLOR_GRAY2BGR)

for y1, y2 in lines:
    cv2.rectangle(output, (0, y1), (output.shape[1], y2), (0, 255, 0), 2)

# ========================
# DISPLAY
# ========================

plt.figure(figsize=(12, 8))

plt.subplot(2, 2, 1)
plt.imshow(gray, cmap="gray")
plt.title("Original")

plt.subplot(2, 2, 2)
plt.imshow(normalized, cmap="gray")
plt.title("Enhanced")

plt.subplot(2, 2, 3)
plt.imshow(clean, cmap="gray")
plt.title("Binary for Segmentation")

plt.subplot(2, 2, 4)
plt.imshow(output)
plt.title("Line Segmentation Output")

plt.tight_layout()
plt.show()
```





```python
# ========================
# STEP 3: Line Segmentation (FIXED ONLY)
# ========================

horizontal_sum = np.sum(clean, axis=1)

threshold = np.max(horizontal_sum) * 0.2
binary = horizontal_sum > threshold

lines = []
start = None

for i in range(len(binary)):
    if binary[i] and start is None:
        start = i
    elif not binary[i] and start is not None:
        end = i

        # 🔥 NEW: split inside region if needed
        segment = horizontal_sum[start:end]

        # find local minima (gaps between lines)
        for j in range(1, len(segment) - 1):
            if segment[j] < segment[j - 1] and segment[j] < segment[j + 1]:
                # split here
                split_y = start + j
                lines.append((start, split_y))
                start = split_y

        lines.append((start, end))
        start = None

if start is not None:
    lines.append((start, len(binary)))
```

4.2 Attempt 2:

Segmentation Refinement

- Modified segmentation logic
- Introduced local minima detection in projection

Outcome

- Minor improvement in splitting merged lines
- Overall instability remained

Insight

- Segmentation failed because the binary image was noisy, not because the segmentation method was incorrect.

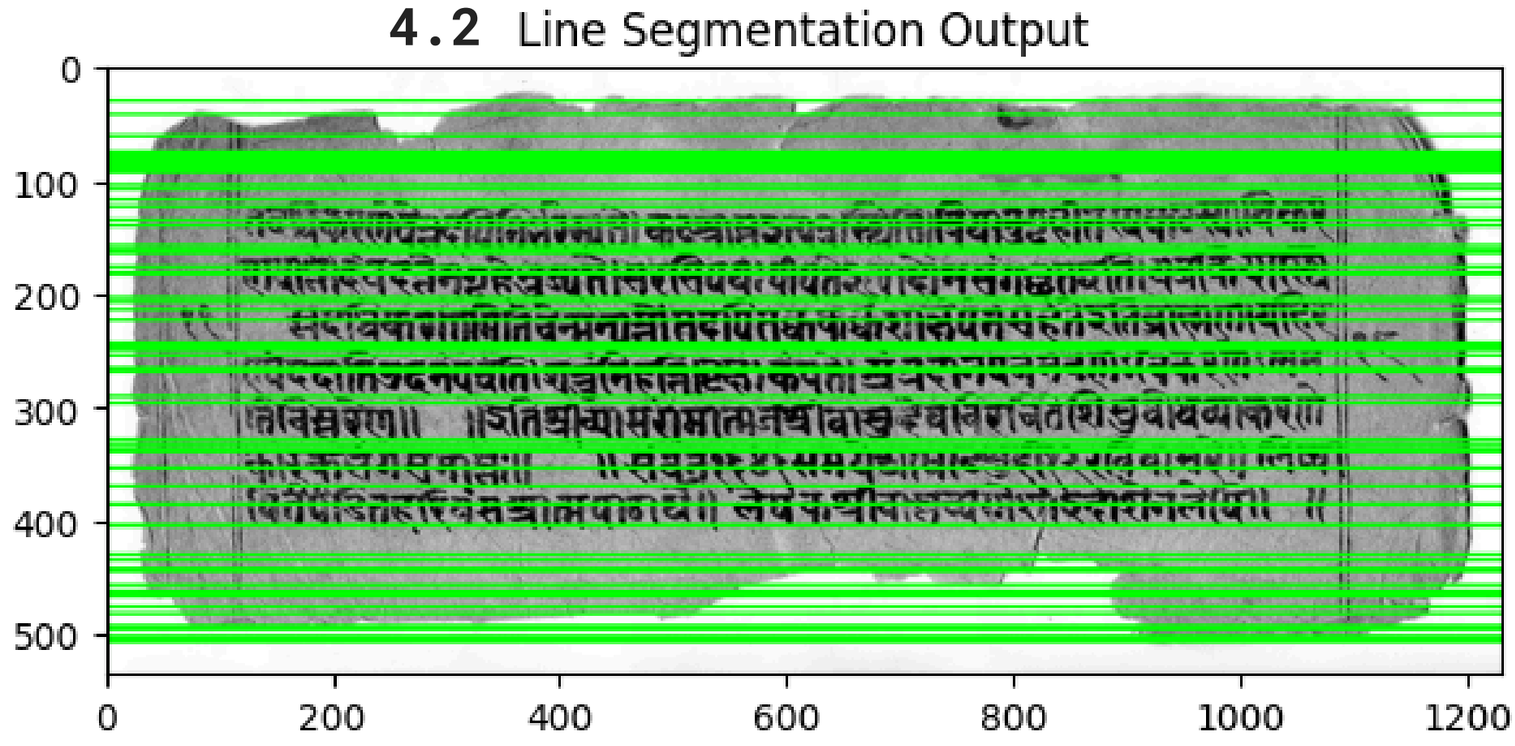
4.3 Attempt 3: Binarization Improvement

changes Introduced

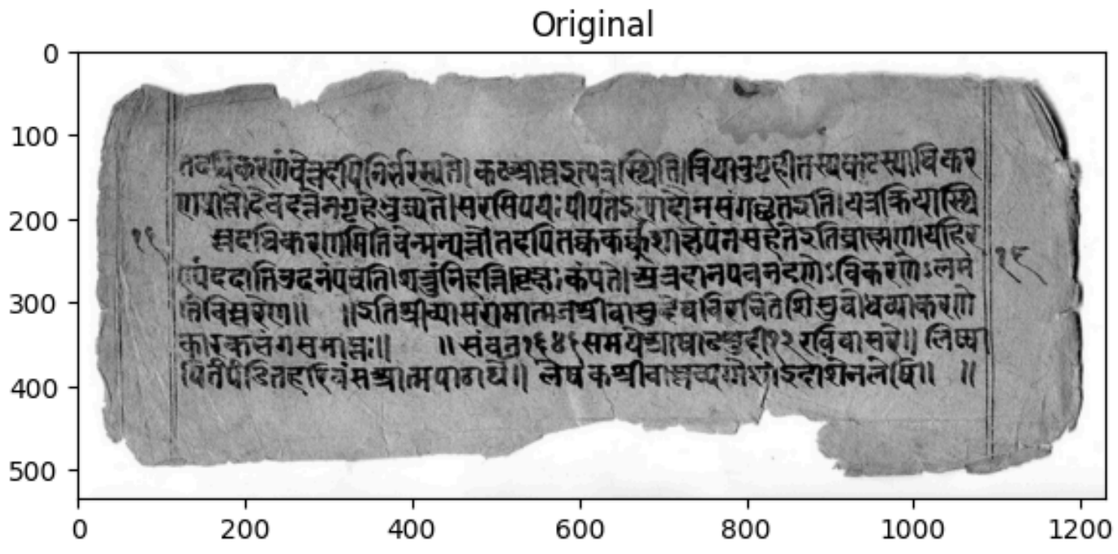
- Strong Gaussian smoothing
- Otsu’s global thresholding
- Morphological cleaning
- Horizontal dilation

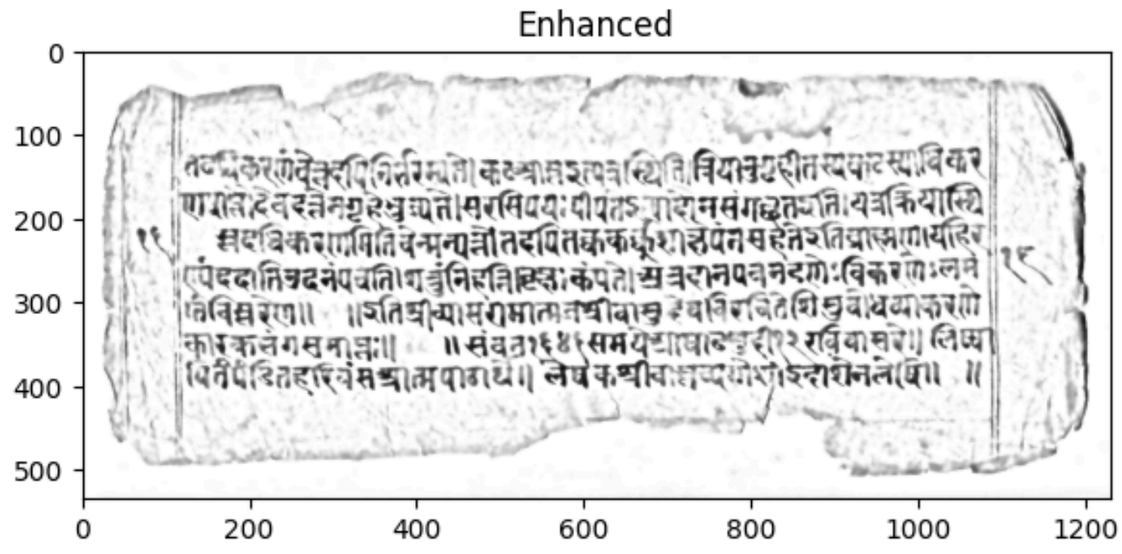
Improvements

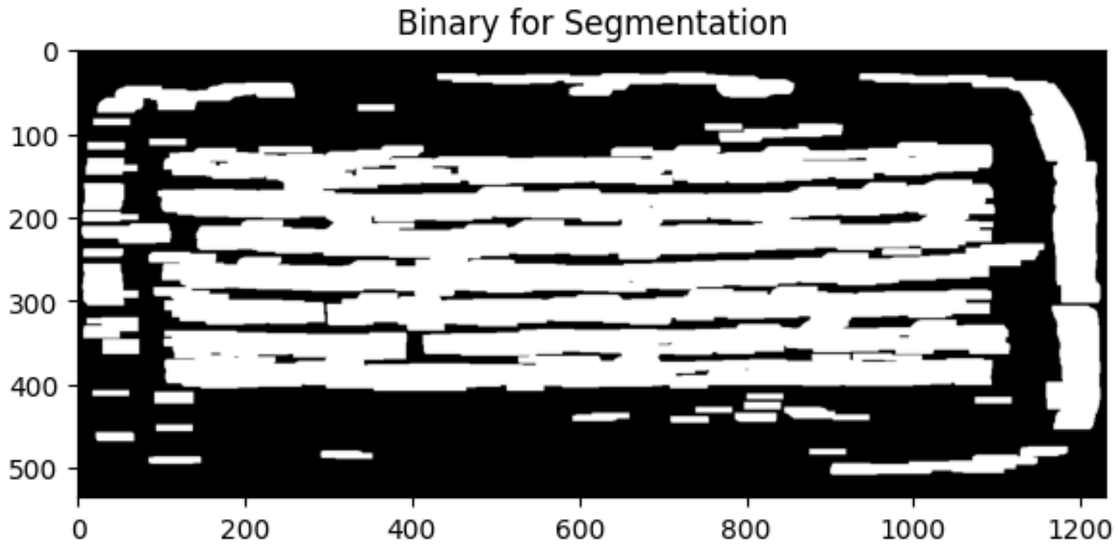
- Significant noise reduction
- Clear separation of text from background
- Text lines became continuous horizontal bands
- Stable projection profile
- Accurate line segmentation
- Minimal merging or fragmentation

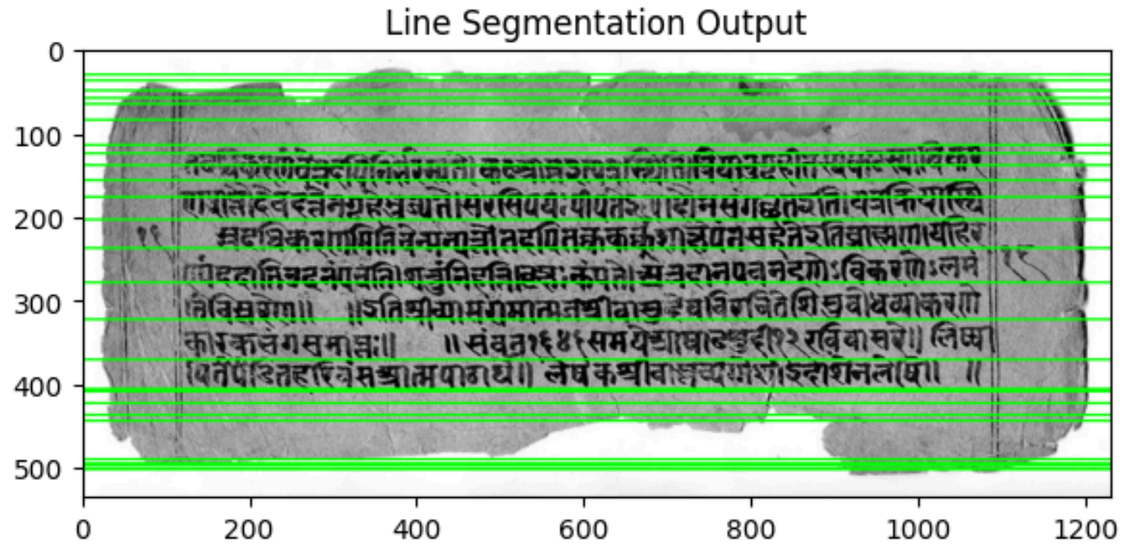



```python
# ========================
# STEP 2: CLEAN BINARIZATION (FIXED)
# ========================

# Strong smoothing to remove texture
blur = cv2.GaussianBlur(normalized, (9, 9), 0)

# Otsu threshold (global, stable)
_, thresh = cv2.threshold(blur, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)

# Remove small noise
kernel = np.ones((3, 3), np.uint8)
clean = cv2.morphologyEx(thresh, cv2.MORPH_OPEN, kernel, iterations=2)

# Connect text horizontally (VERY IMPORTANT)
kernel_line = cv2.getStructuringElement(cv2.MORPH_RECT, (40, 3))
clean = cv2.dilate(clean, kernel_line, iterations=1)
```



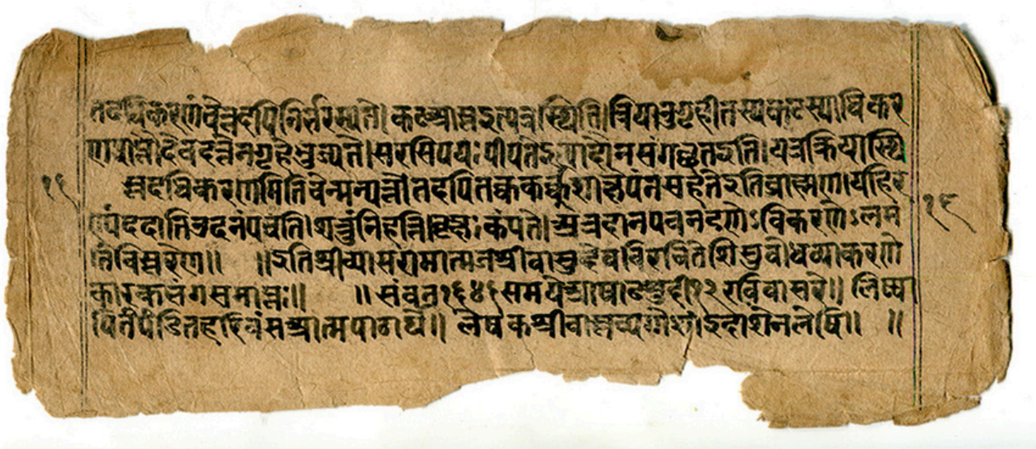






Final Pipeline

- Grayscale conversion
- CLAHE (contrast enhancement)
- Bilateral filtering
- Background normalization
- Gaussian smoothing
- Otsu thresholding
- Morphological opening
- Horizontal dilation
- Projection-based line segmentation



Key Observation

- Improving binarization had a greater impact on segmentation accuracy than modifying segmentation algorithms
- CLAHE: Improves local contrast by redistributing intensity values within small regions, making faint text more visible.

Conclusion

This project demonstrates that classical image processing techniques can effectively enhance and segment palm leaf manuscripts when applied systematically. The iterative refinement process highlighted the importance of preprocessing, particularly binarization, in achieving reliable results. The final pipeline is interpretable, reproducible, and suitable for further applications such as OCR.

Conceptual Understanding

1. Grayscale Conversion
The input image is first converted to grayscale so that the processing becomes simpler. Since the main goal is to separate text from background based on intensity, color information is not necessary. Working in grayscale also reduces computation and avoids unnecessary complexity.

2. CLAHE (Contrast Enhancement)
CLAHE is used to improve the visibility of faint text. In palm leaf manuscripts, some characters are very light and difficult to distinguish. CLAHE enhances contrast locally, meaning it works on small regions instead of the whole image. This helps in bringing out hidden text without over-brightening the entire image.

3. Bilateral Filtering
After contrast enhancement, some noise also becomes visible. Bilateral filtering is applied to reduce this noise while keeping the edges of the characters intact. This is important because we don’t want the text boundaries to get blurred during smoothing.

4. Background Normalization
The background of palm leaf manuscripts is uneven due to natural texture and lighting. To fix this, a blurred version of the image is used to estimate the background, and then the image is normalized using this background. This step helps in making the background more uniform and improves the distinction between text and background.

5. Gaussian Smoothing
Even after normalization, some texture still remains. A Gaussian blur is applied to further smooth out small variations and reduce unwanted patterns from the leaf surface. This makes the next step (thresholding) more reliable.

6. Otsu’s Thresholding
Instead of using adaptive thresholding, Otsu’s method is used to convert the image into binary form. It automatically finds a threshold that separates text and background. In this case, it works better because adaptive methods were treating background texture as text, which caused noise.

7. Morphological Opening
The binary image still contains small unwanted pixels. Morphological opening is used to remove these tiny noise elements while keeping the actual text intact. This results in a cleaner binary image.

8. Horizontal Dilation
At this stage, some characters in the same line may still be slightly disconnected. Horizontal dilation is applied to connect these characters so that each text line forms a continuous region. This step is very important for correct line detection.

9. Horizontal Projection
To detect text lines, the number of white pixels in each row is calculated. Rows with a higher number of white pixels correspond to text regions, while rows with fewer pixels represent spaces between lines.

10. Line Segmentation
Finally, a threshold is applied to the projection values to identify text lines. Continuous regions above the threshold are grouped together and marked as individual lines. These lines are then highlighted on the original image.